OsdagBridge: Parametric 3D CAD Modeling of Bridge Substructure and IFC Workflow Integration

FOSSEE Osdag Screening Task Report

1. Methodology

The substructure modeling for OsdagBridge uses a procedural, parametric geometry generation approach built on pythonOCC (OpenCASCADE Python bindings). The implementation is organized into five modular builder functions, each responsible for a specific reinforced-concrete component, following the same procedural pattern (keyword-only parameters, private underscore-prefixed helpers, and a dict[str, list[TopoDS_Shape]] return type) already established by the existing superstructure builders in the repository.

Pier and Pile Builders

Piers and piles are both modeled as circular columns using BRepPrimAPI_MakeCylinder, positioned with a gp_Ax2 axis placed at the correct base coordinate and extruded vertically along +Z. Piles are instantiated in a 2x2 grid (4 per pier) beneath each pile cap, with 600 mm center-to-center spacing.

Pile Cap Builder

The pile cap is a simple rectangular prism generated with BRepPrimAPI_MakeBox, sized 2200 mm x 1200 mm x 600 mm and centered beneath the pier base, enveloping the pile grid.

Pier Cap Builder

The pier cap is modeled as a tapered trapezoidal prism (hammerhead profile). A trapezoidal cross-section is defined in the X-Z plane using an ordered set of gp_Pnt vertices (top width 3000 mm, bottom width 1200 mm), built into a wire and face via BRepBuilderAPI_MakeWire and BRepBuilderAPI_MakeFace, and then extruded along the transverse Y-axis with BRepPrimAPI_MakePrism. The pier cap length auto-derives from the total deck width computed by the generator rather than being hardcoded, satisfying the task's parametric requirement.

Reinforcement Builder

A single reusable build_reinforcement() function generates 3D rebar cages for the pier, pier cap, and pile cap. It produces main longitudinal/vertical bars (16 mm diameter, 150 mm spacing) and transverse ties (8 mm diameter, 200 mm spacing) as cylindrical solids, offset inward from the outer concrete surface by a 40 mm cover distance.

Pipeline Integration

All five builders are orchestrated by PlateGirderCADGenerator.generate() in cad_generator.py, which computes global placement coordinates (span-end abutment positions at X = 0 and X = span_length, centered at Y = 0) from the overall design parameters and calls each builder in turn. The aggregated shape dictionary is passed to the presentation layer in cad_3d.py, where each shape list is wrapped as an AIS_Shape, registered against the viewer's model_ais_objects and component-visibility map, and displayed with the appropriate color and transparency.

2. IFC4 BIM Integration

Interoperability with external BIM tools is achieved by extending the existing IfcOpenShell-based export pipeline (PlateGirderIFCExtractor -> BridgeGeometryMapper -> BridgeIfcGenerator) to include the new substructure elements alongside the superstructure.

Spatial Hierarchy and Semantic Mapping

A dedicated IfcBuildingStorey named "Substructure" was added beneath the existing IfcBuilding, separate from the superstructure's storey. Each component type was mapped to its correct semantic IFC4 class:

- Pier -> IfcColumn
- Pier Cap -> IfcBeam
- Pile Cap -> IfcFooting
- Pile -> IfcPile
- Rebar -> IfcReinforcingBar

Geometry and Placement

Circular cross-sections (piers, piles) are represented natively in IFC via IfcCircleProfileDef extruded through IfcExtrudedAreaSolid, rather than tessellated OCC BREP geometry, keeping the export lightweight and parametric — consistent with how the existing superstructure elements are exported. Each substructure element carries an IfcLocalPlacement defined by an IfcAxis2Placement3D, parented to the Substructure storey's placement. This preserves a well-defined global coordinate system (X = longitudinal span, Y = transverse deck width, Z = vertical) while allowing each component's local placement to be relative to its own origin, matching the convention already used for the superstructure. This ensures that when the IFC file is opened in external BIM software (e.g., Revit), all substructure geometry aligns correctly beneath the superstructure with no floating or overlapping elements, and quantities can be correctly extracted for a Bill of Quantities.

3. Challenges and Resolutions

Environment Setup and Dependency Conflicts

A significant portion of development time was spent resolving local environment instability before any substructure code could even be tested end-to-end:

- **Broken environment / PATH conflicts:** The project's designated conda environment (osdagbridge-env) was found to be incompletely installed. A separate, working environment (parabridge) was located and used instead, and a standalone system Python install taking priority in PATH over conda meant `conda activate` silently failed to switch environments; commands had to be run using the full explicit path to the environment's python.exe.
- **PySide6 / shiboken6 DLL corruption:** The GUI crashed on startup with a native "Windows fatal exception: access violation" during import of a compiled Qt resource module. Diagnosis (via import-time tracing) revealed a corrupted, partially-uninstalled leftover PySide6 folder shadowing the real installation. This was resolved with a clean forced reinstall of PySide6 and shiboken6 (--force-reinstall --no-deps), removing the stale directory first.
- **Missing osdag_core dependency:** The application imports from an osdag_core package that is not published as a standard pip package. The conda-channel osdag package initially installed did not

expose the expected osdag_core module structure (cli, design_type, Common, etc.). The correct fix was to clone the official Osdag repository's dev branch directly from GitHub and install it in editable mode (pip install -e .) into the working environment.

OpenCASCADE Rendering Bugs in cad_3d.py

Two subtle rendering bugs surfaced in the interactive 3D viewer, despite the underlying geometry being verified as mathematically correct (confirmed independently via bounding-box inspection of the exported IFC solids).

Transparency and hidden rebar

Concrete components (pier, pier cap, pile cap, piles) rendered as fully opaque white in the live interactive GUI, completely hiding the internal rebar, even though the correct transparency value (0.35) was being computed and passed into the display helper function. The initial attempt passed the transparency value as a keyword argument directly into display.DisplayShape(shp, transparency=..., **kwargs); this proved unreliable and was silently ignored by the OpenGL rendering backend on this system, because the shape's default material did not support alpha blending at the point DisplayShape() was called.

The working fix was to first create the AIS_Shape via DisplayShape() without any transparency argument, and then explicitly call ais.SetTransparency(float(transparency)) directly on the returned AIS_Shape object, followed by context.RecomputePrsOnly(ais, False) to force the viewer to recompute and redraw the shape's presentation with the new alpha value. This two-step approach — display first, then set transparency on the live AIS object — proved to be the reliable pattern, and was verified visually in the real interactive GUI (not a headless render) after the fix.

Pier cap taper rendering as a box

The trapezoidal pier cap initially rendered as a plain rectangular block with no visible taper, even though the generated vertex coordinates for the cross-section were dimensionally correct (top width 3000 mm, bottom width 1200 mm). Investigation traced this to the winding order of the gp_Pnt vertices defining the X-Z cross-section: when ordered clockwise, the resulting face normal pointed opposite to the extrusion direction used by BRepPrimAPI_MakePrism, producing a topologically inconsistent solid that the viewer's rendering backend fell back to displaying as its bounding box. Reordering the vertices to a consistent counter-clockwise winding (matching the intended +Y extrusion direction) resolved the issue and restored the correct tapered profile in the interactive view.

4. References

- pythonOCC / OpenCASCADE documentation — https://dev.opencascade.org/doc/overview/html/index.html
- IfcOpenShell documentation — https://docs.ifcopenshell.org/
- Osdag core repository — https://github.com/osdag-admin/Osdag
- OsdagBridge repository — https://github.com/osdag-admin/OsdagBridge